gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843 Continuous Integration

Dr. Francisco Gomes de Oliveira Neto (he/him)

Lynn Conway

- Revolutionised the design of microchips
- Caused the “Mead–Conway VLSI chip design revolution” and reshaped Electrical Engineering and Comp. Science education.
- Tech trailblazer and transgender pioneer

What we will learn today

- What, Why and How of **Continuous Integration**? How to **introduce it**?
- What is **Docker** and how does it work?
- What are the differences between **Virtual Machines and Containers**?
- What is **Continuous Delivery** and **Deployment**? What are **feature flags**?
- What are **Canary Releases**? What is **AB Testing**? When should we use them?

© Francisco Oliveira Neto - 2018

Part 1:
Configuration management and Continuous Integration

© Francisco Oliveira Neto - 2022

Where it all begins…

- Configuration management

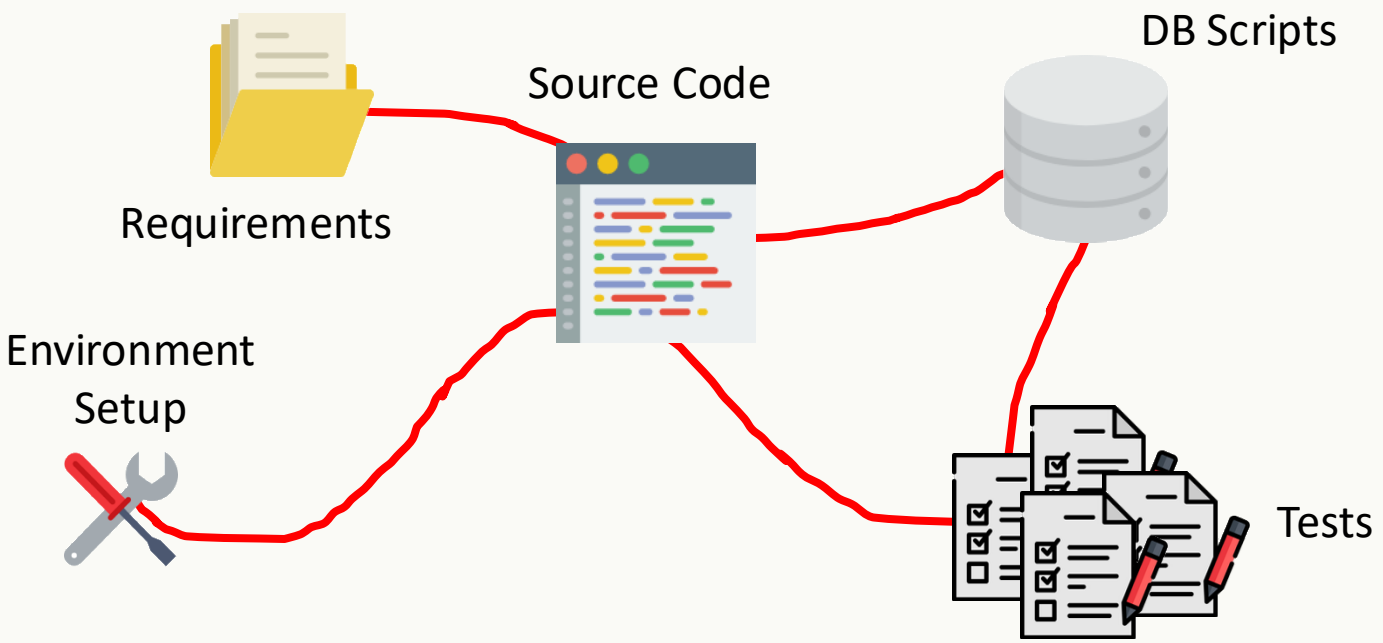
*… Refers to the process by which **all artefacts** relevant to your project and **relationships between them**, are stored, **retrieved**, uniquely identified and modified.*

From: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation Humble, Jez; Farley, David. Page 31.

© Francisco Oliveira Neto - 2018

Configuration Management

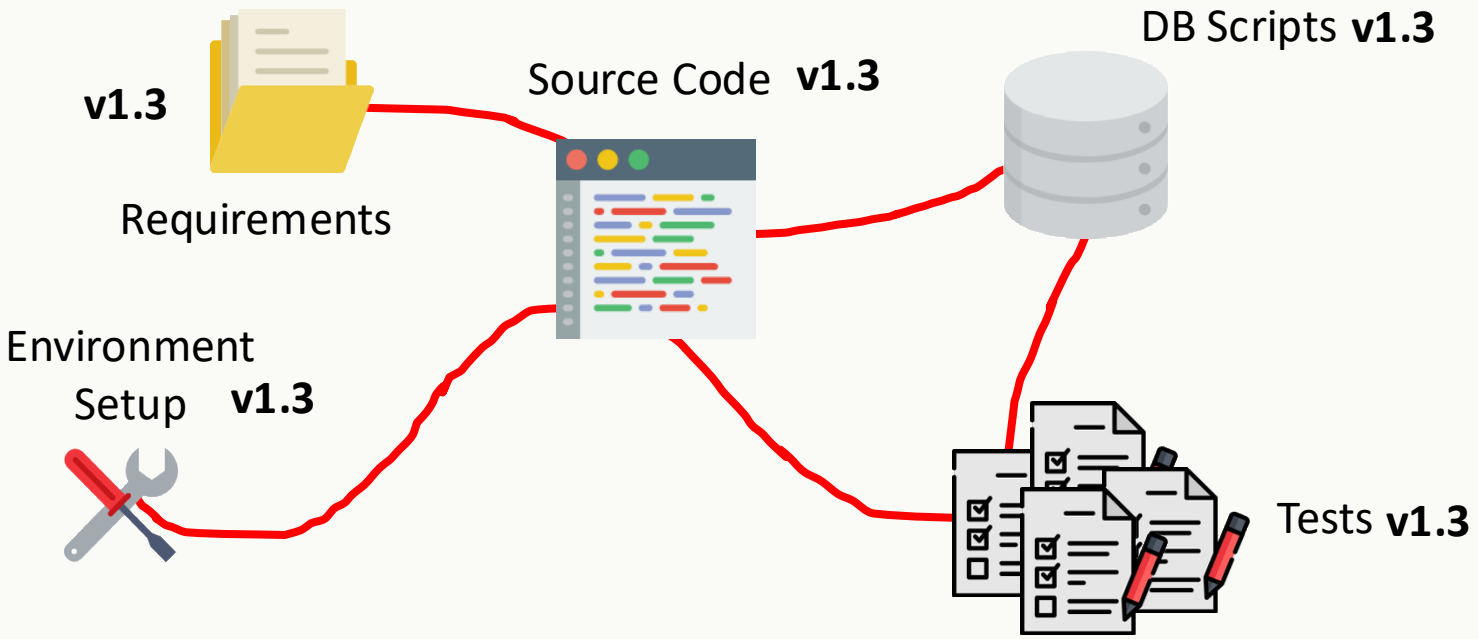
- Have a **strategy** for storing baselines and controlling changes for:



© Francisco Oliveira Neto - 2018

Configuration Management

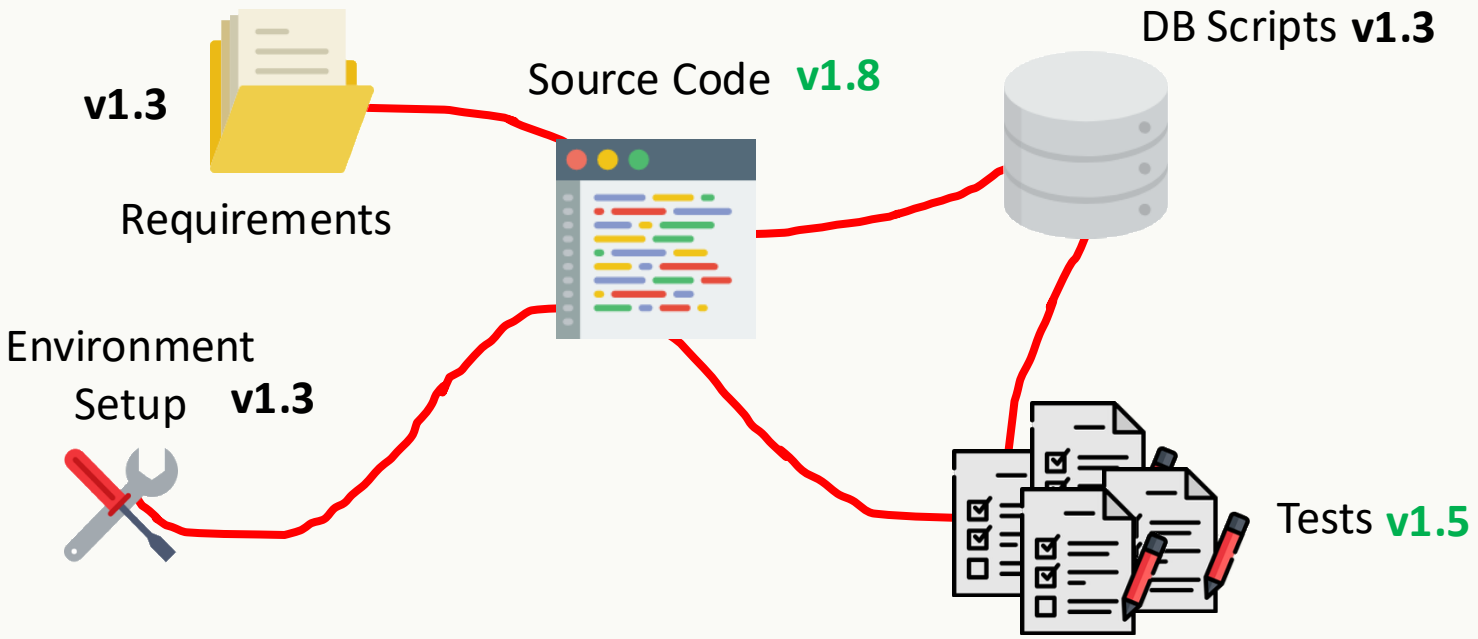
- Have a **strategy** for storing baselines and controlling changes for:



© Francisco Oliveira Neto - 2018

Configuration Management

- Have a **strategy** for storing baselines and controlling changes for:



© Francisco Oliveira Neto - 2018

Configuration Management

- It’s not just a question of choosing and implementing a tool, **it’s bigger than that**.

- Enable:
  - **Control and awareness** of your application’s state.
  - **Reproducibility** of setup and deployment
  - Strategically exploit **version-controlled assets**

© Francisco Oliveira Neto - 2018

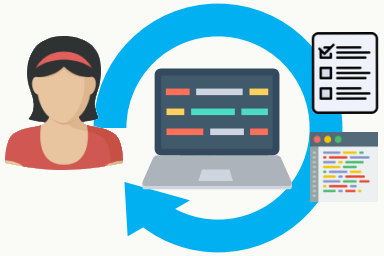
Continuous Integration - What

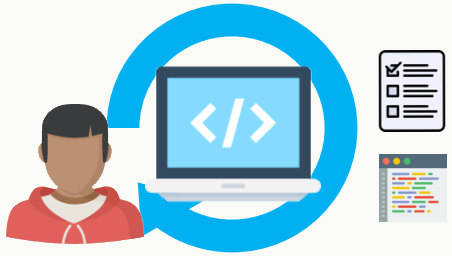
Continuous Integration requires that every time ***somebody commits any change,*** *the* ***entire application is built*** *and a comprehensive set of automated* ***tests is run against it****.*

From: Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation Humble, Jez; Farley, David. Page 55.

© Francisco Oliveira Neto - 2018

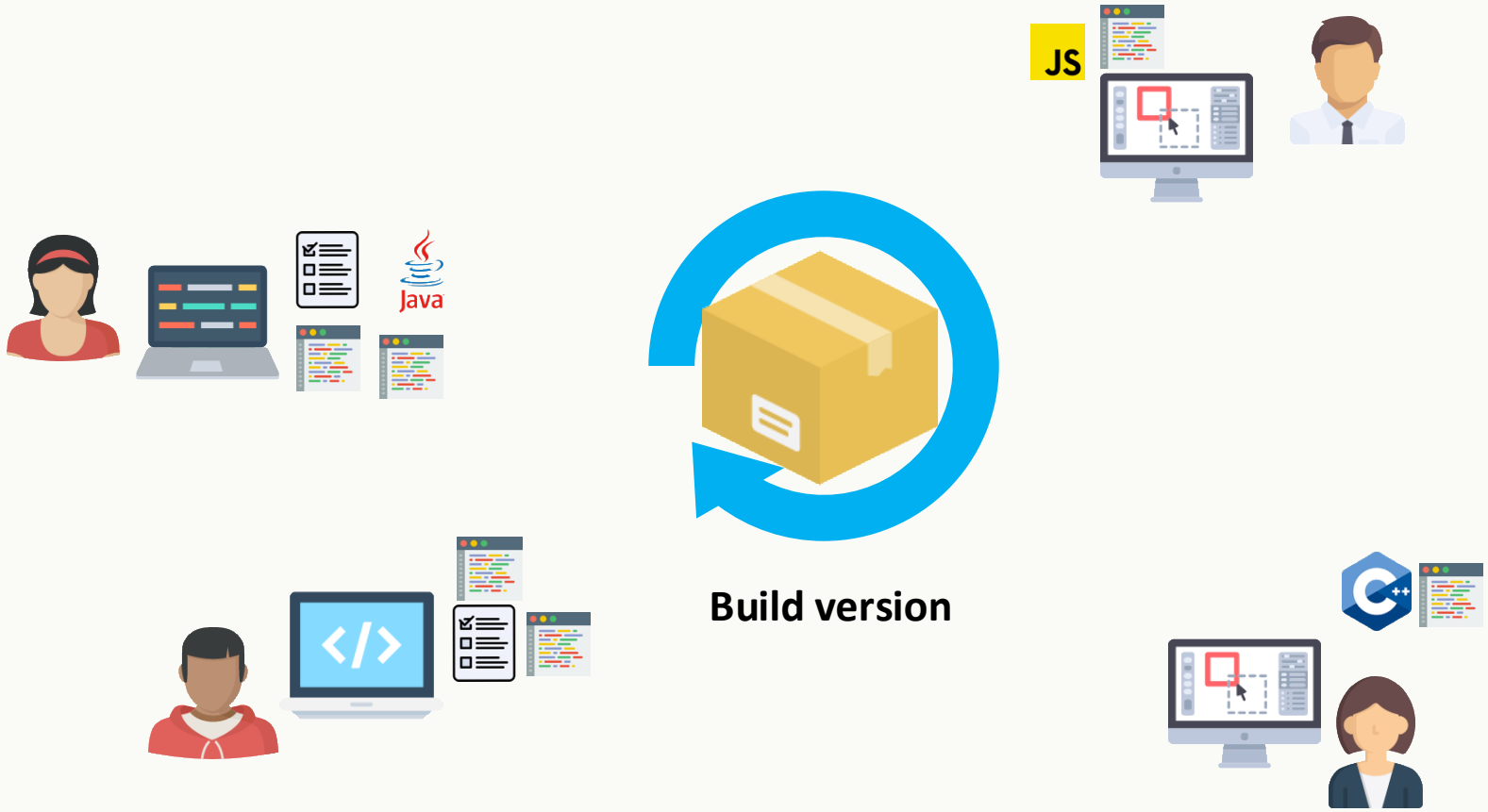
Continuous Integration - Before





- Developers working with their **code locally**.
- **No systematic way** to test the integration of changes made by different developers.
- **First steps** were in continuous testing practices.
- Continuous testing means **running automated tests** as you modify small parts of the code.
- If you don't do CI, you will hear the "… But it runs on my machine." excuse too often. **Avoid that**.

Continuous Integration - After



Continuous Integration - Why

It’s a **paradigm shift**!

- Integration happens more **systematically**.
- Developers submit their changes to a **shared code repository**.
- **Run tests** on the **integrated version**. Identify changes that break the integration.

© Francisco Oliveira Neto - 2018

How to implement it?

- **Version** control (Git, SVN, etc.)

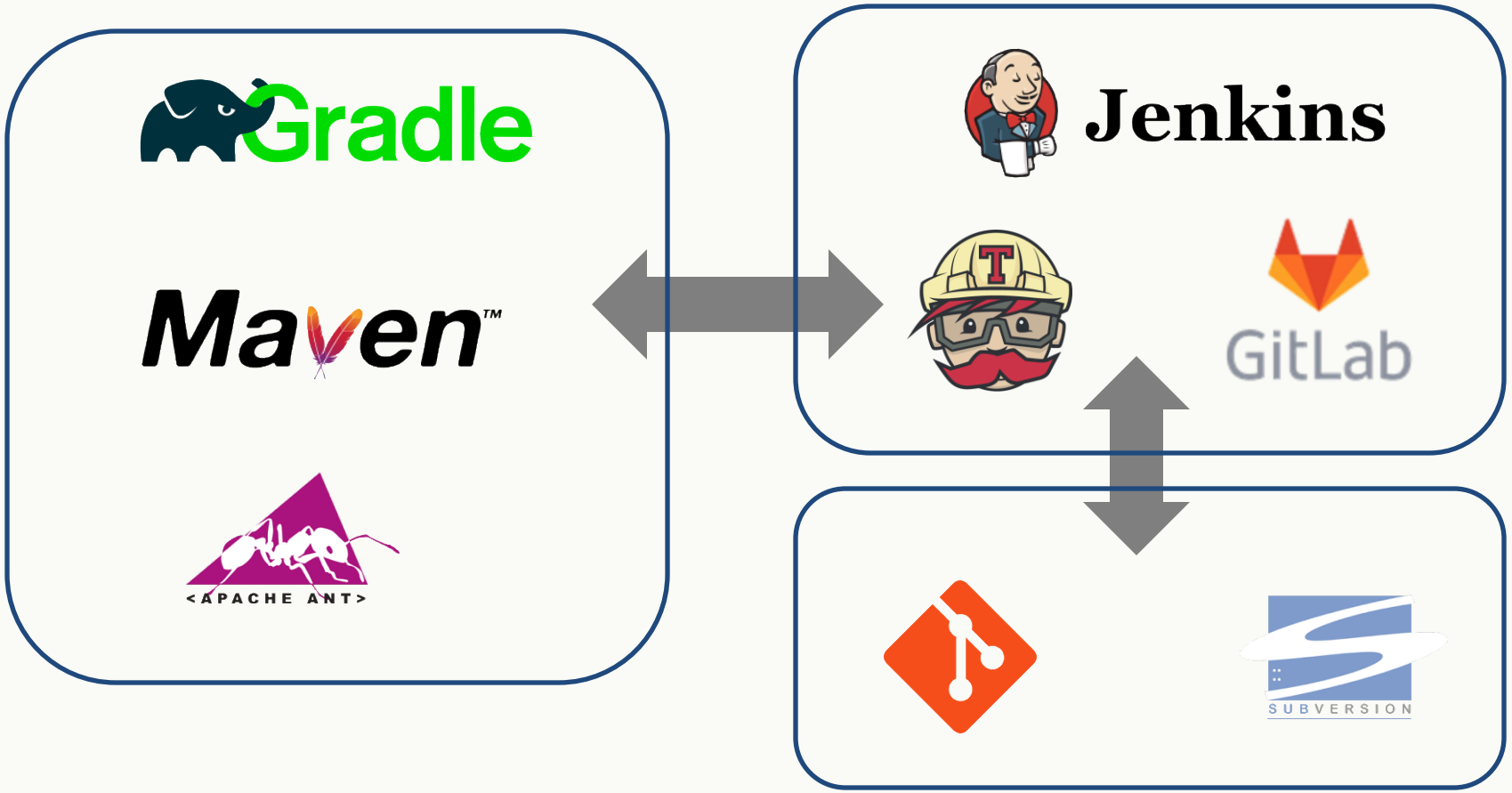
- **Automated** build

- **Agreement** of the team

© Francisco Oliveira Neto - 2018

How to implement it

Automate Project Build



Automation Server

(Cloud)

Version Control

Source Code

(Repository)

© Francisco Oliveira Neto - 2018

From: https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment

Important Requirements for CI

- Your team will need to **write and run automated tests** for each new feature, improvement or bug fix.
- You need a **continuous integration server** that can monitor the main repository.
- Developers need to **merge their changes as often as possible**, at least once a day.

© Francisco Oliveira Neto - 2018

Trade-offs related to CI

- Limitations:
  - **Infrastructure** maintenance and **dependency.**
  - Requires **diligence** in applying the **good practices.**
- Benefits:
  - **Testing costs are reduced drastically, due to less manual work.**
  - **Continuous feedback** allows for easier debugging due to small “delta” between versions.
  - **Replaces** a lot of **manual** and ad-hoc procedures. Less context switching as developers are alerted to failures.

© Francisco Oliveira Neto - 2018

Part 1: Take-aways

- Continous management is a practice where all **modifications done to all artefacts** can be tagged, stored, retrieved, and restored.

- Continous integration is a practice where the software changes are **incorporated and tested frequently** and on a dedicated environment.

- CI requires version control, automation server, build tools and discipline.
- Continous and automated testing allows for **easier debugging** and regression testing.

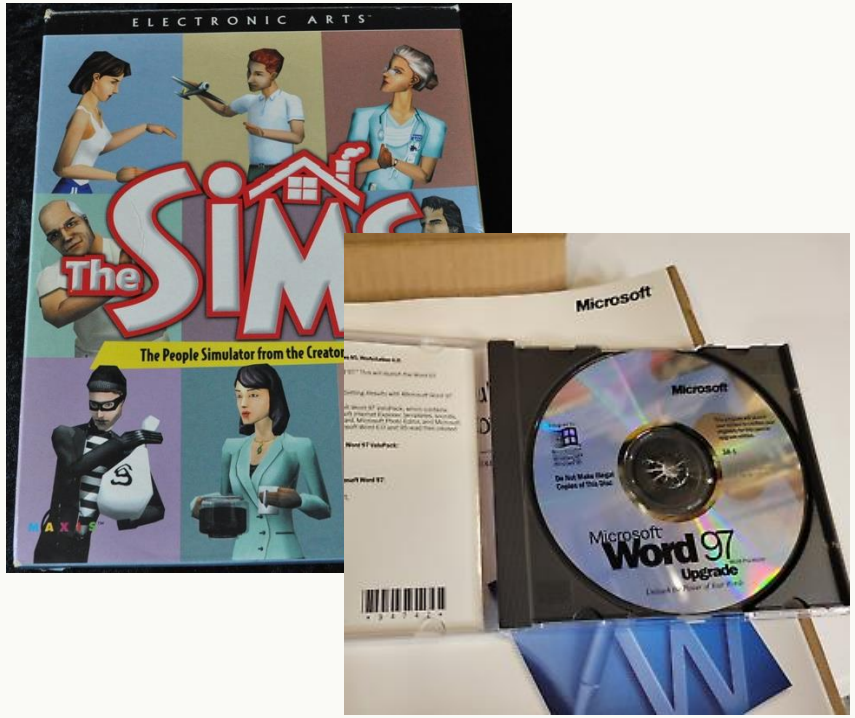
© Francisco Oliveira Neto - 2022

Part 2:

Software and Platform as a Service

© Francisco Oliveira Neto - 2022

Software as a Service



- Before, we used to **buy physical media** to install and run software.

- Cost-effective Internet and Architecture allowed us to communicate seamlessly with **online servers in real-time.**

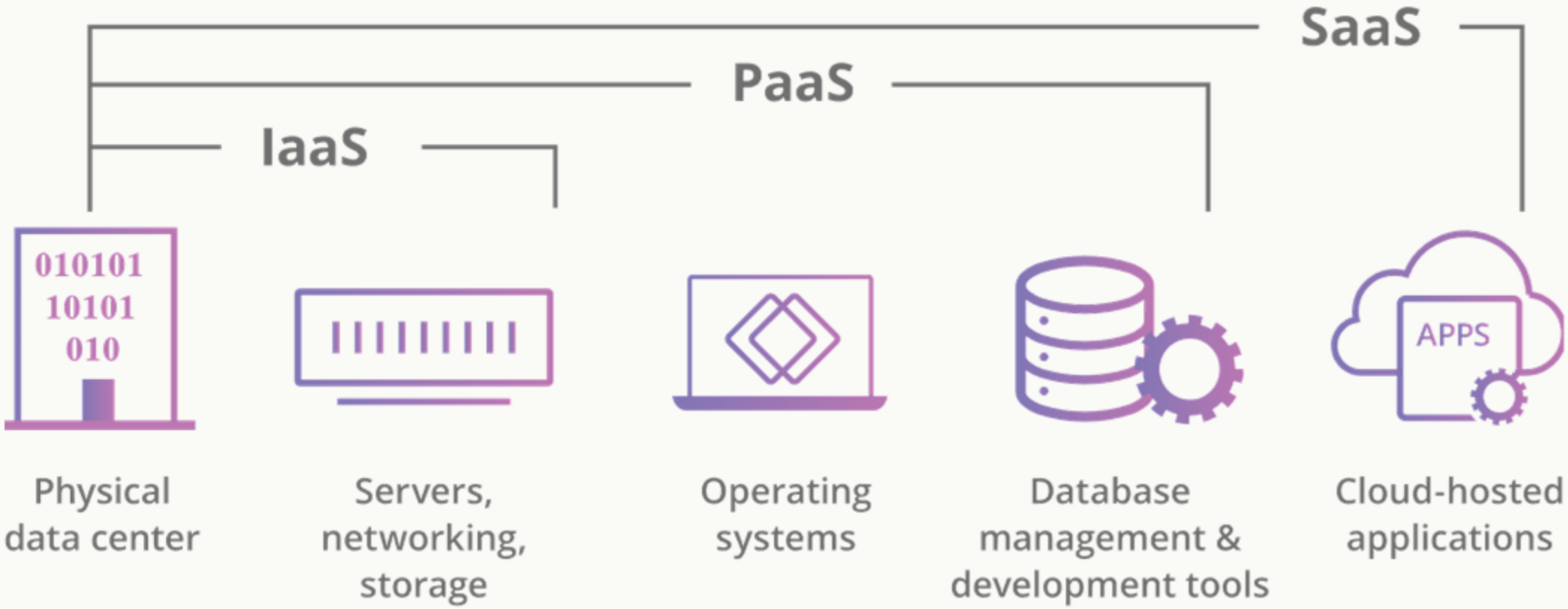
Platform as a Service - PaaS

- **Platform as a Service (PaaS):** Cloud computing services that offer a platform to manage modular **bundles of software applications**. Then we can offer Software as a Service (SaaS).

- **Docker** became popular by implementing the PaaS concept to provide **Operating System-level virtualisation**. In other words, a platform (Docker Engine) that offers the base infrastructure for an operating system.

- Before Docker, we used **virtual machines** to split and **manage the resources of a hardware platform**. For instance, a machine with 250 GB hard drive could “loan” 100 GB for a virutal machine.

Platform as a Service - PaaS



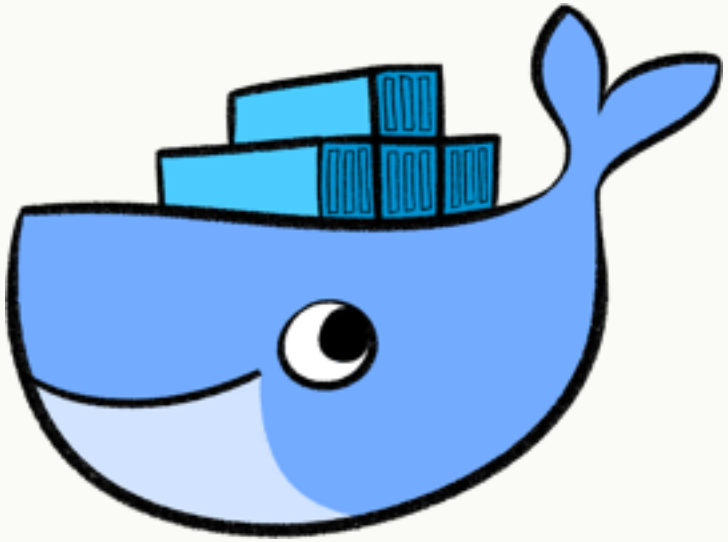
IT WORKS ON MY MACHINE
THEN WE'LL SHIP YOUR MACHINE
AND THAT IS HOW DOCKER WAS BORN

Docker – Containers and Platform

- Typically, a **platform** is Hardware + Operating System of a computer. This is the **bare bones** of a computer.

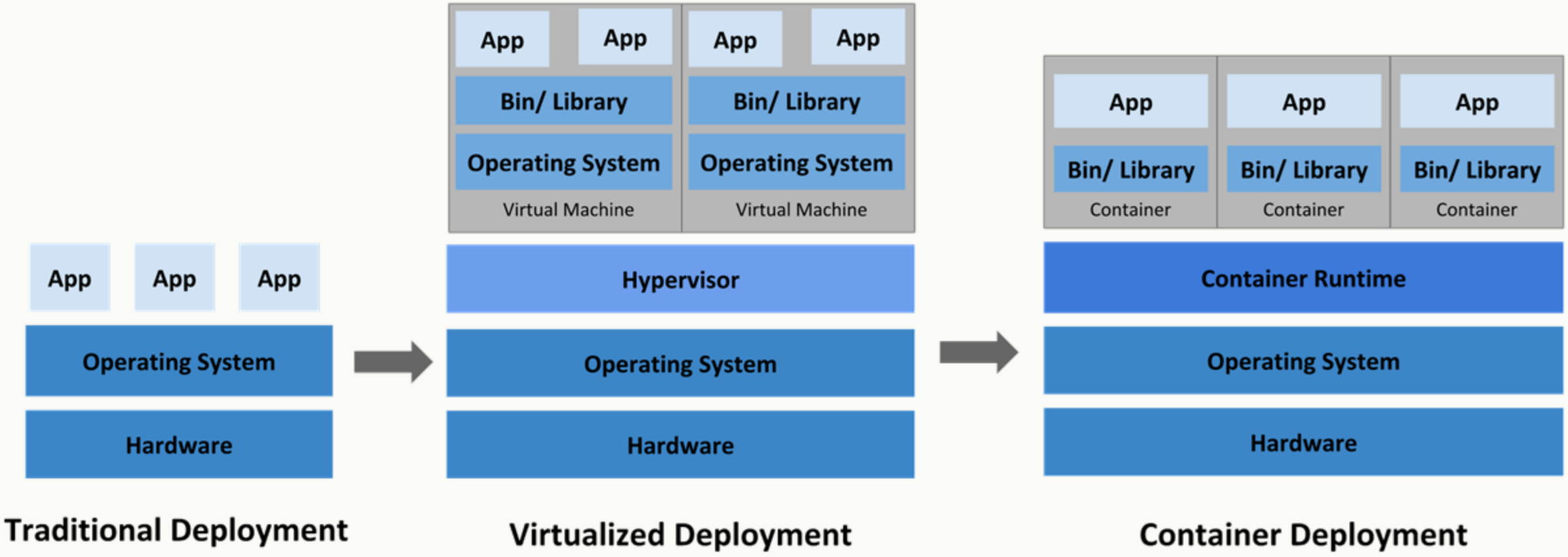
- **Docker Engine**, creates a platform by using the local resources to manage **“isolated” software applications**.

- The **bundled applications** running on those platforms are called Containers. This is very useful to handle **compatibility** of software.



Src: https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/

Docker vs. Virtual Machines



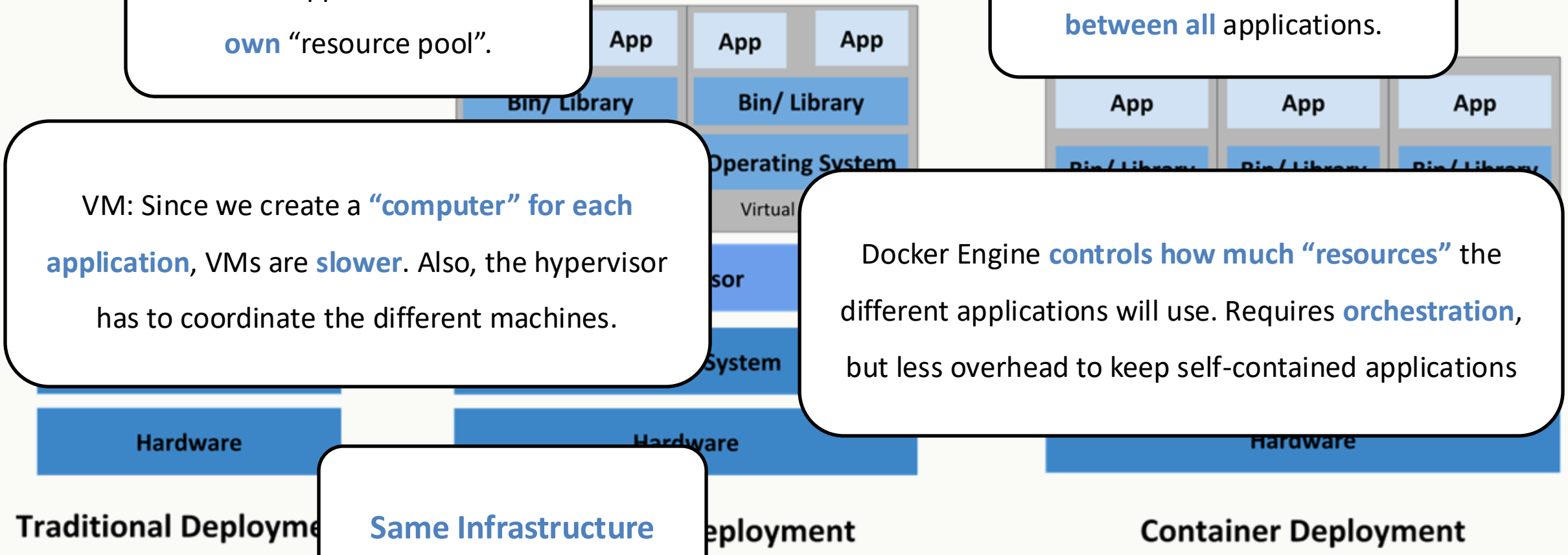
Docker vs. Virtual Machines

VM: Each application has **their own** “resource pool”.

The “resource pool” is **shared between all** applications.

VM: Since we create a **“computer” for each application**, VMs are **slower**. Also, the hypervisor has to coordinate the different machines.

Docker Engine **controls how much “resources”** the different applications will use. Requires **orchestration**, but less overhead to keep self-contained applications



Same Infrastructure

Src: https://www.docker.com/blog/supporting-open-source-projects-at-docker/whale-logo332_5/

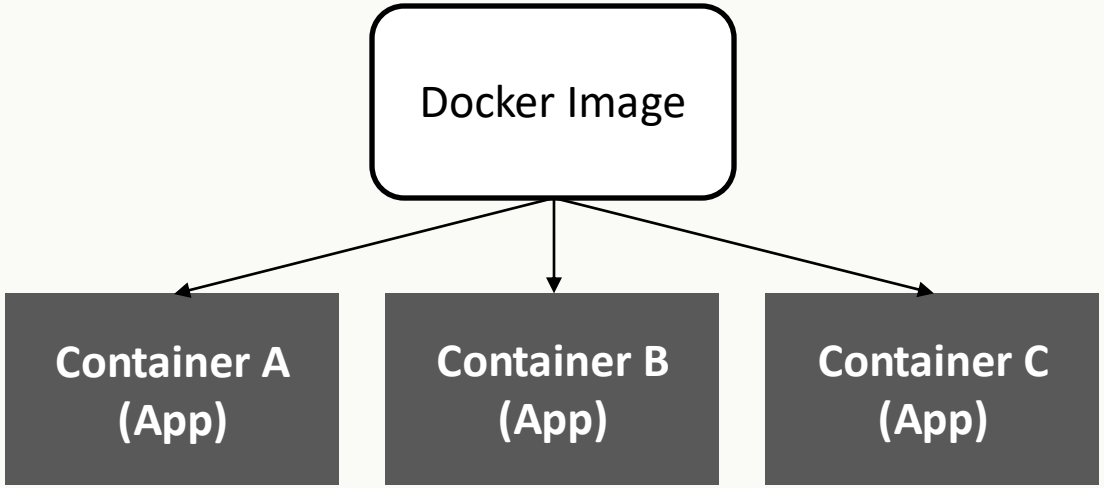
Docker – Important Concepts

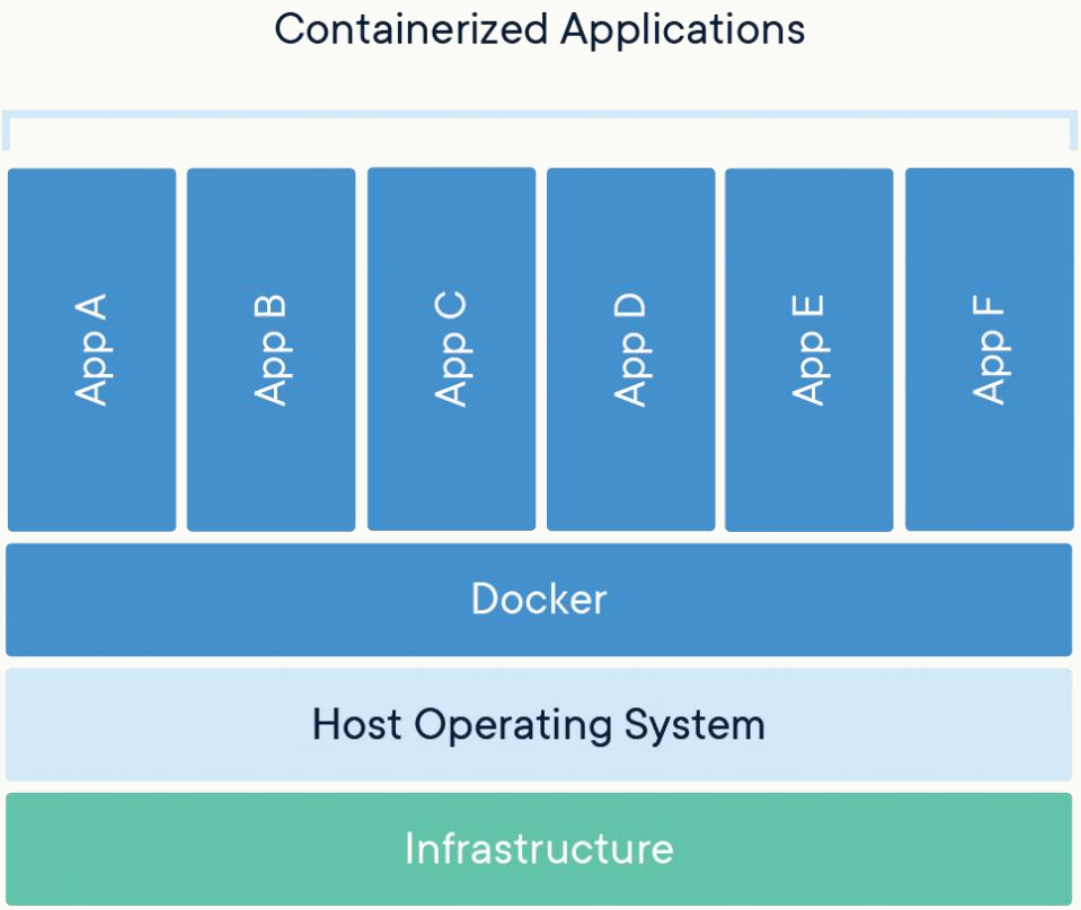
- **Image:** Lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.

- **Container:** Executable application created from an image. Runs on Docker Engine.

- **Docker Engine:** The platform that shares and handles resource allocation between the computer (memory, processing power, etc.) and the running application.

Docker and Containers

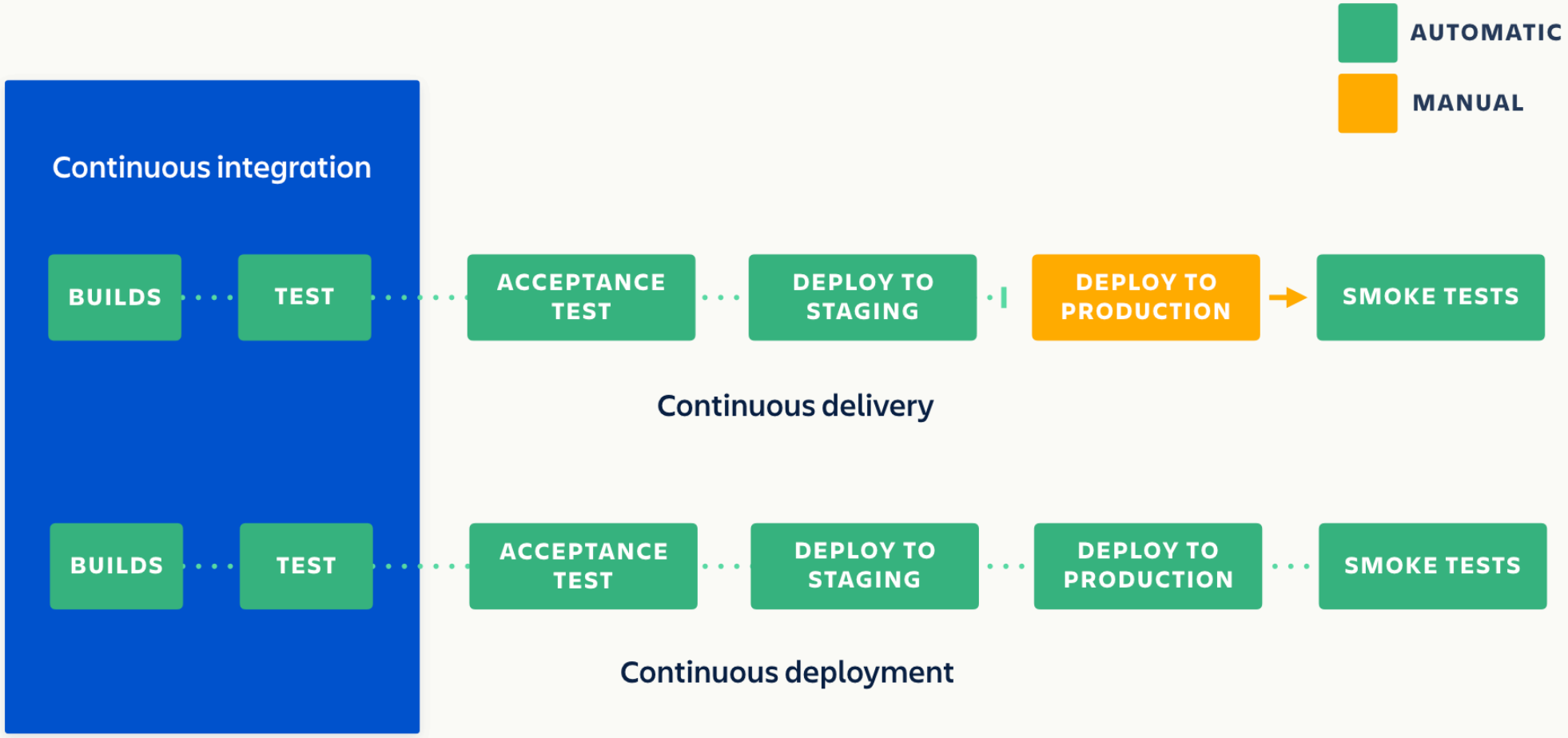




Part 3:
Continous Delivery
and
Continous Deployment

© Francisco Oliveira Neto - 2022

Continuous Integration vs. Delivery vs. Deployment



© Francisco Oliveira Neto - 2022

Continuous Delivery

- **Continuous delivery** is an extension of continuous integration. Automatically deploys all code changes to a **testing and/or production environment** after the build stage.
- This means that on top of automated testing, you have an **automated release process** and you can deploy your application any time by clicking a button.
- **In theory**, with continuous delivery, you can decide to release daily, weekly, fortnightly, or whatever suits your business requirements.
- **Need to embrace feature flags** so that incomplete features do not affect customers in production.

© Francisco Oliveira Neto - 2022

Feature Flags – What and Why

- **Feature flags** (feature toggles) is a software engineering technique that turns select **functionality on and off** during runtime, **without deploying new code**.

- More controlled experimentation over the lifecycle of features.
- Because of this, feature flags enable many **novel workflows** that are useful to an agile management style and CI/CD environments.

© Francisco Oliveira Neto - 2022

From: https://docs.getunleash.io/reference/sdks/java

Feature Flags – How

- **Wrap** desired code paths in a feature flag. Example (Java Unleash framework):

```
if(unleash.isEnabled("AwesomeFeature")) {
    //do some magic
} else {
    //do old boring stuff
}
```

- Yes, under the hood, feature flags are essentially just "if statements".
- Still, you want **frameworks to help manage flag** data or injection and removal of the new logic path.

© Francisco Oliveira Neto - 2022

Continuous Deployment

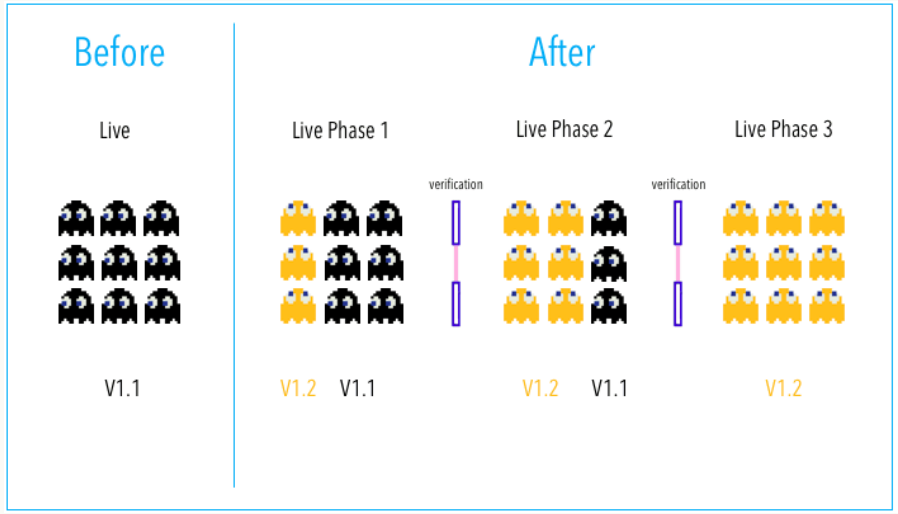
- Every change that passes all stages of your production pipeline is **released to your customers**.
- There's **no human intervention**, and only a failed test will prevent a new change to be deployed to production.
- Continuous deployment is an excellent way to accelerate the **feedback loop with your customers**. See their work go live minutes after they've finished working on it.
- Your **testing culture needs to be at its best**. The quality of your test suite will determine the quality of your releases.

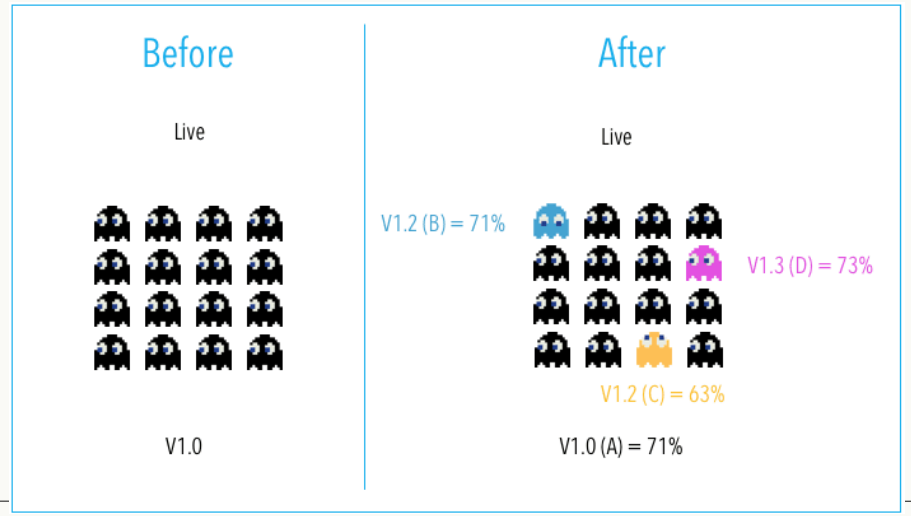
© Francisco Oliveira Neto - 2022

Img: https://harness.io/blog/blue-green-canary-deployment-strategies/

Deployment Strategies

- **Canary releases:** Incremental releases of the application to subset of users. **Gradually** increase the percentage of users through phases. Low risk but requires control to enable rollback.
- **AB Testing:** Simultaneous deploy varied versions of the same application to **experiment on end-users**. Main benefit is to allow exploration with users. Only one (or none) of the versions will be kept. Popular for changing UI design.





Part 3: Take-aways

- CD is a step above from CI because the deployment stage must be done automatically, i.e., automated release process (to production).

- There are frameworks for feature flagging where we can **automatically toggle features** in or out of a product's release.

- **AB Testing and Canary** releases are different techniques that harness the benefit of Continous Deployment.

© Francisco Oliveira Neto - 2022